



DATA STRUCTURES

(Course Code: 25B11CI318)

Overview

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

Need for Linked Lists

Instead of storing elements in contiguous memory, we store each element in a separate memory location.

Each element stores

- Data
- Address of the next node

Need for Linked Lists

Instead of storing elements in contiguous memory, we store each element in a separate memory location.

Each element stores

- Data
- Address of the next node

Advantages

- Dynamic Size
- Easy Insertion
- Easy Deletion
- Better Memory Utilization

What is a Linked List?

Definition

A linked list is a linear data structure in which each element is called a **node**.

Each node contains

- ① Data
- ② Address of the next node

The nodes are connected using pointers.

What is a Linked List?

Definition

A linked list is a linear data structure in which each element is called a **node**.

Each node contains

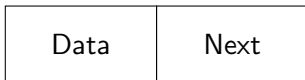
- ① Data
- ② Address of the next node

The nodes are connected using pointers.

$$Node = (Data, Next)$$

Structure of a Node

Node Representation

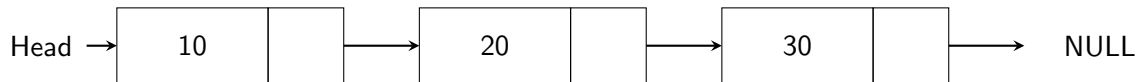


C Structure

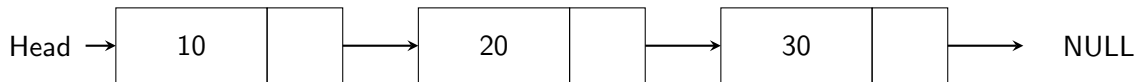
```
1 struct Node
2 {
3     int data;
4     struct Node *next;
5 };
```

The pointer stores the address of the next node.

Representation of a Linked List



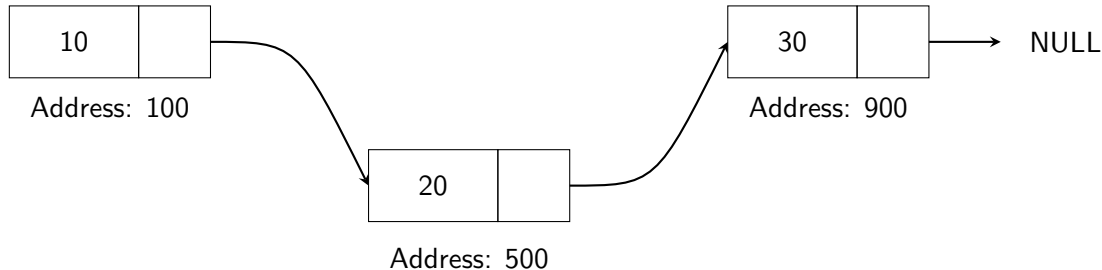
Representation of a Linked List



Nodes are connected using pointers.
The last node points to **NULL**.

Memory Representation of a Linked List

Unlike arrays, linked list nodes are stored at **different memory locations**.

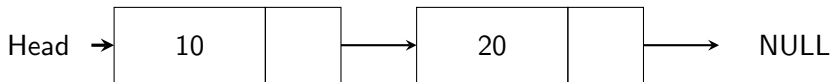


Observation

Nodes need not be stored in contiguous memory.

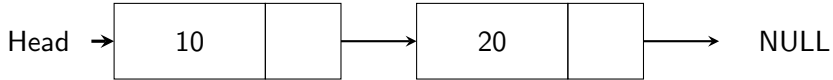
Head Pointer

The **Head Pointer** stores the address of the first node in the linked list.



Head Pointer

The **Head Pointer** stores the address of the first node in the linked list.



Important

If

Head = NULL

then the linked list is empty.

Dynamic Memory Allocation

Nodes are created during program execution.

```
#include <stdlib.h>

struct Node *newNode;

newNode =
(struct Node*)
malloc(sizeof(struct Node));
```

Dynamic Memory Allocation

Nodes are created during program execution.

```
#include <stdlib.h>

struct Node *newNode;

newNode =
(struct Node*)
malloc(sizeof(struct Node));
```

Memory is allocated from the

Heap Memory

Dynamic Memory Allocation

Nodes are created during program execution.

```
#include <stdlib.h>

struct Node *newNode;

newNode =
(struct Node*)
malloc(sizeof(struct Node));
```

Memory is allocated from the

Heap Memory

Advantages

- Memory allocated only when required.
- No fixed size limitation.

Creating a New Node

```
1 struct Node *newNode;  
2  
3 newNode =  
4 (struct Node*)  
5 malloc(sizeof(struct Node));  
6  
7 newNode->data = 25;  
8  
9 newNode->next = NULL;
```

Creating a New Node

```
1 struct Node *newNode;  
2  
3 newNode =  
4 (struct Node*)  
5 malloc(sizeof(struct Node));  
6  
7 newNode->data = 25;  
8  
9 newNode->next = NULL;
```

Result

25	NULL
----	------

Traversal of a Linked List

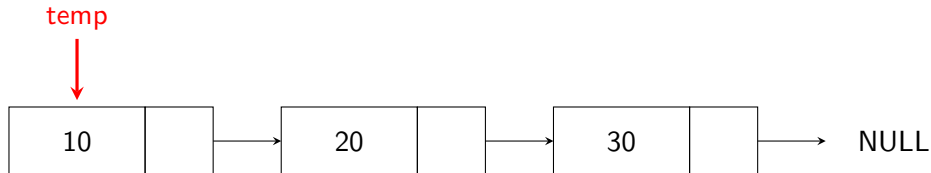
Algorithm

```
temp = head;
while(temp != NULL)
{
    printf("%d ", temp->data);
    temp = temp->next;
}
```

Traversal of a Linked List

Algorithm

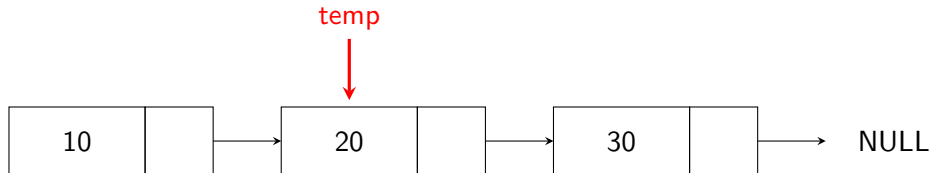
```
temp = head;  
  
while(temp != NULL)  
{  
    printf("%d ",temp->data);  
  
    temp = temp->next;  
}
```



Traversal of a Linked List

Algorithm

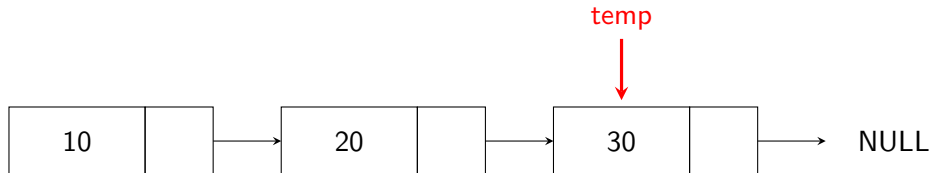
```
temp = head;  
  
while(temp != NULL)  
{  
    printf("%d ",temp->data);  
  
    temp = temp->next;  
}
```



Traversal of a Linked List

Algorithm

```
temp = head;  
  
while(temp != NULL)  
{  
    printf("%d ",temp->data);  
  
    temp = temp->next;  
}
```



Accessing Nodes

Unlike arrays,

No Direct Indexing

Accessing Nodes

Unlike arrays,

No Direct Indexing

To access the 4th node, we must visit

$$1^{st} \rightarrow 2^{nd} \rightarrow 3^{rd} \rightarrow 4^{th}$$

Accessing Nodes

Unlike arrays,

No Direct Indexing

To access the 4th node, we must visit

$1^{st} \rightarrow 2^{nd} \rightarrow 3^{rd} \rightarrow 4^{th}$

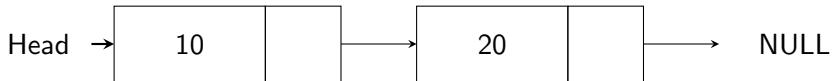
Operation	Complexity
Access	$O(n)$
Traversal	$O(n)$
Search	$O(n)$

Key Point

Linked Lists provide sequential access, not random access.

Insertion at the Beginning

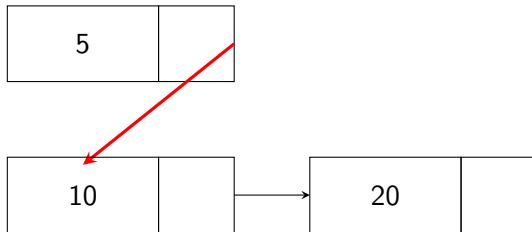
Goal: Insert **5** before the first node.



Insertion at the Beginning

Goal: Insert **5** before the first node.

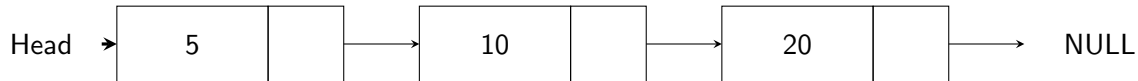
New Node



Time Complexity = $O(1)$

Insertion at the Beginning

Goal: Insert **5** before the first node.



Time Complexity = $O(1)$

Insertion at the End

Insert node containing **40**.

Steps

Insertion at the End

Insert node containing **40**.

Steps

- 1 Traverse to the last node.

Insertion at the End

Insert node containing **40**.

Steps

- ① Traverse to the last node.
- ② Create a new node.

Insertion at the End

Insert node containing **40**.

Steps

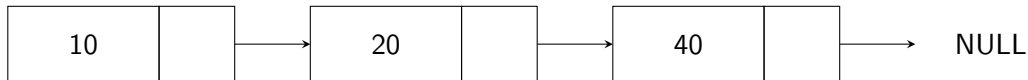
- ① Traverse to the last node.
- ② Create a new node.
- ③ Last node's next points to new node.

Insertion at the End

Insert node containing **40**.

Steps

- 1 Traverse to the last node.
- 2 Create a new node.
- 3 Last node's next points to new node.
- 4 New node points to NULL.

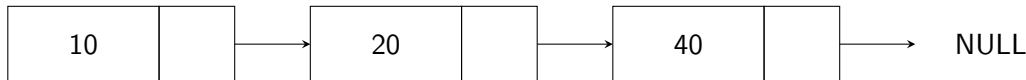


Insertion at the End

Insert node containing **40**.

Steps

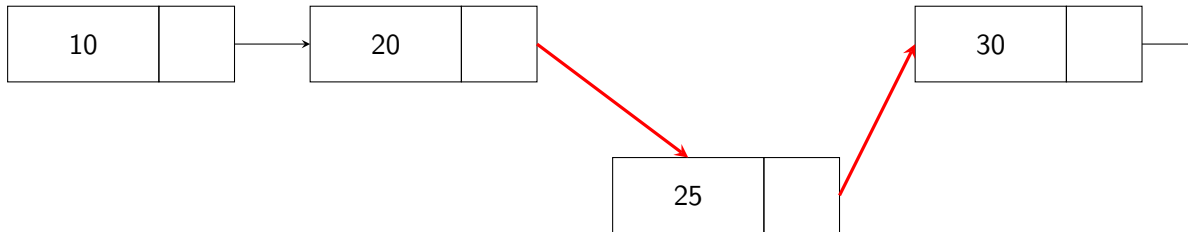
- 1 Traverse to the last node.
- 2 Create a new node.
- 3 Last node's next points to new node.
- 4 New node points to NULL.



$O(n)$

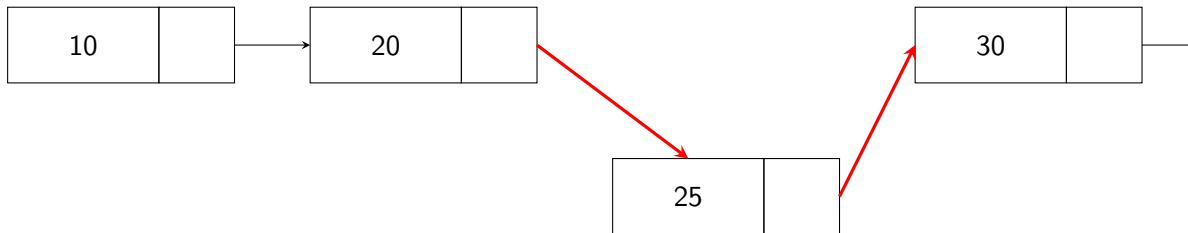
Insertion After a Given Node

Insert **25** after node containing 20.



Insertion After a Given Node

Insert **25** after node containing 20.



Two pointer updates are required.

$newNode \rightarrow next = temp \rightarrow next$

$temp \rightarrow next = newNode$